

Unit 6: Image Segmentation

Yuba Raj Devkota | NCCS

Image Processing | CSIT 5

Unit 6	Image Segmentation	Teaching Hours (8)
Introduction	Definition, Similarity and Discontinuity based techniques	1 hr
Discontinuity Based Techniques	Point Detection, Line Detection, Edge Detection using Gradient and Laplacian Filters, Mexican Hat Filters, Edge Linking and Boundary Detection, Hough Transform	3 hrs.
Similarity based techniques	Thresholding: Global, Local and Adaptive Region Based Segmentation: Region Growing Algorithm, Region Split and Merge Algorithm	4 hrs.

Image Segmentation

- *Image Segmentation is a process in image processing where an image is divided into multiple meaningful regions or objects. The goal is to simplify the image so that it becomes easier to analyze.*
- *In segmentation, pixels with similar characteristics (such as color, intensity, or texture) are grouped together.*

Image segmentation is the process of partitioning an image into different regions so that each region represents a meaningful part of the image.

Example:

- Separating a person from the background
- Detecting tumors in medical images
- Identifying roads in satellite images

Applications:

- Medical imaging
- Object detection
- Face recognition
- Autonomous vehicles
- Satellite image analysis

Similarity vs Discontinuity Based Segmentation Techniques

Basis	Similarity Technique	Discontinuity Technique
Concept	Groups similar pixels	Detects sudden intensity changes
Focus	Homogeneous regions	Boundaries or edges
Methods	Thresholding, Region Growing	Edge Detection, Line Detection
Goal	Region formation	Boundary detection

- *Similarity-based techniques group pixels with similar properties.*
- *Discontinuity-based techniques detect sharp changes to find boundaries.*

Discontinuity Based Segmentation Technique

- *Discontinuity-based techniques divide an image based on sudden changes in intensity or color.*
- *Instead of similarity, these methods detect edges or boundaries where pixel values change sharply.*

Common Discontinuity Methods

1. Edge Detection

Detects boundaries between objects.

Common operators:

- Sobel Operator
- Prewitt Operator
- Canny Edge Detector
- Roberts Operator

2. Line Detection

Detects straight lines in the image.

3. Point Detection

Detects isolated points that are significantly different from surrounding pixels.

Key Idea:

Segmentation is performed by identifying sharp intensity changes (edges).

Point Detection

Point detection is used to identify **isolated pixels** whose intensity is significantly different from their surrounding pixels.

In an image, a point is detected when a pixel intensity value is **much brighter or darker than its neighboring pixels**.

Concept

A **mask (filter kernel)** is applied to the image. The mask highlights pixels that have a strong intensity difference compared to their neighbors.

A commonly used mask for point detection is:

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

The mask is applied over the image using **convolution**. If the resulting value is **greater than a specified threshold**, that pixel is considered a detected point.

Simple Image Example

Consider a small 3×3 **image region** where the center pixel is much brighter than the surrounding pixels.

Image (pixel intensity values):

$$\begin{bmatrix} 10 & 10 & 10 \\ 10 & 80 & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

Here:

- Neighbor pixels = 10
- Center pixel = 80

This indicates a possible **isolated bright point**.

Mask Used for Point Detection

A common **point detection mask** is:

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

This mask highlights pixels whose value is **very different from their neighbors**.

Now calculate:

Neighbor contribution:

$$8 \times (-10) = -80$$

Center contribution:

$$8 \times 80 = 640$$

Total result:

$$640 - 80 = 560$$

Convolution Calculation

Multiply each element of the image with the corresponding mask value and sum the result.

$$\begin{aligned} & (-1 \times 10) + (-1 \times 10) + (-1 \times 10) \\ & + (-1 \times 10) + (8 \times 80) + (-1 \times 10) \\ & + (-1 \times 10) + (-1 \times 10) + (-1 \times 10) \end{aligned}$$

Threshold Decision

If the output value is **greater than a predefined threshold**, the pixel is detected as a **point**.

Example:

Threshold = **100**

Since

$$560 > 100$$

the center pixel is **detected as an isolated point**.

Final Result

The algorithm marks the center pixel as a **detected point** because its intensity is significantly higher than its neighbors.

Detected output:

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Here 1 represents the detected point.

Example

Consider detecting **stars in a night sky image**.

- The sky background has low intensity.
- A star appears as a **bright isolated pixel**.
- Point detection filters identify these bright points.

Applications

- Detecting **stars in astronomical images**
- Identifying **defects in industrial inspection**
- Detecting **noise points in digital images**

Line Detection

Line detection is used to detect **thin lines present in an image**. Lines may appear in different orientations such as **horizontal, vertical, or diagonal**.

This technique uses specific **convolution masks** designed to detect lines in a particular direction.

Line detection is used to detect lines present in an image. These lines can be horizontal, vertical, or diagonal. The detection is done using directional masks (filters) that highlight lines in a particular orientation.

Example

Consider detecting **road markings in a satellite image**.

- Road markings appear as **bright lines**.
- A line detection filter highlights these structures.
- The output image shows the detected lines clearly.

Applications

- **Road and lane detection** in autonomous vehicles
- **Fingerprint analysis**
- **Medical image analysis** (blood vessel detection)
- **Document analysis** (text line detection)

Simple Image Example

Suppose we have a 3×3 **image region** where a horizontal line exists.

Image (pixel intensity values):

$$\begin{bmatrix} 10 & 10 & 10 \\ 80 & 80 & 80 \\ 10 & 10 & 10 \end{bmatrix}$$

Here:

- Middle row pixels are **bright (80)**
- Other pixels are **dark (10)**

This represents a **horizontal bright line**.

Convolution Calculation

Multiply corresponding elements and sum the values.

$$\begin{aligned} &(-1 \times 10) + (-1 \times 10) + (-1 \times 10) \\ &+ (2 \times 80) + (2 \times 80) + (2 \times 80) \\ &+ (-1 \times 10) + (-1 \times 10) + (-1 \times 10) \end{aligned}$$

Now calculate:

Mask for Horizontal Line Detection

To detect a **horizontal line**, the following mask is used:

$$\begin{bmatrix} -1 & -1 & -1 \\ 2 & 2 & 2 \\ -1 & -1 & -1 \end{bmatrix}$$

This mask emphasizes the **middle row**, which helps detect horizontal structures.

Top row:

$$-10 - 10 - 10 = -30$$

Middle row:

$$160 + 160 + 160 = 480$$

Bottom row:

$$-10 - 10 - 10 = -30$$

Total result:

$$480 - 60 = 420$$

Threshold Decision

If the result is **greater than a predefined threshold**, a line is detected.

Example:

Threshold = **100**

Since

$$420 > 100$$

the system detects a **horizontal line**.

Detected Output

The center row is identified as a **line region**.

$$\begin{bmatrix} 0 & 0 & 0 \\ 1 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

Here **1** indicates detected line pixels.

Common Line Detection Masks

Horizontal line detection

$$\begin{bmatrix} -1 & -1 & -1 \\ 2 & 2 & 2 \\ -1 & -1 & -1 \end{bmatrix}$$

Diagonal line detection (45°)

$$\begin{bmatrix} -1 & -1 & 2 \\ -1 & 2 & -1 \\ 2 & -1 & -1 \end{bmatrix}$$

Vertical line detection

$$\begin{bmatrix} -1 & 2 & -1 \\ -1 & 2 & -1 \\ -1 & 2 & -1 \end{bmatrix}$$

Diagonal line detection (135°)

$$\begin{bmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{bmatrix}$$

Edge Detection using Gradient and Laplacian Filters

Feature	Gradient Filter	Laplacian Filter
Derivative type	First derivative	Second derivative
Direction	Detects edges in specific directions	Detects edges in all directions
Operators	Sobel, Prewitt, Roberts	Laplacian
Sensitivity	Less sensitive to noise	More sensitive to noise

- **Gradient edge detection** finds edges by measuring the **first derivative (intensity change)** in horizontal and vertical directions.
- **Laplacian edge detection** uses the **second derivative** to detect edges in all directions.

Edge Detection using Gradient Filters

Gradient-based edge detection detects edges by measuring the **rate of change of intensity** in an image. If the intensity changes rapidly between neighboring pixels, it indicates the presence of an **edge (boundary of an object)**.

The gradient of an image is calculated in two directions:

- **Horizontal direction (G_x)**
- **Vertical direction (G_y)**

Common gradient operators include **Sobel, Prewitt, and Roberts operators**.

Consider a small 3×3 image region where the right side is brighter than the left side.

$$\begin{bmatrix} 10 & 10 & 200 \\ 10 & 10 & 200 \\ 10 & 10 & 200 \end{bmatrix}$$

Sobel Mask (Vertical Edge Detection)

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

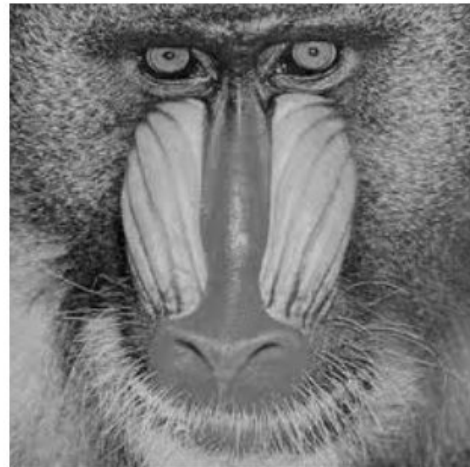
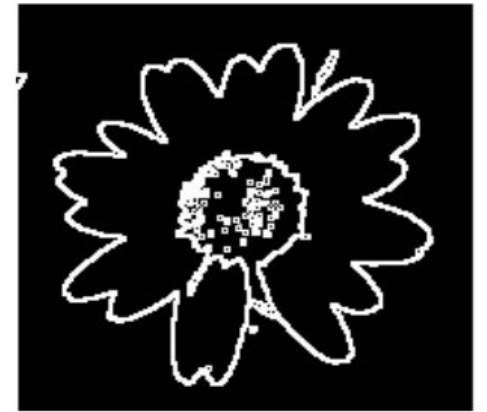
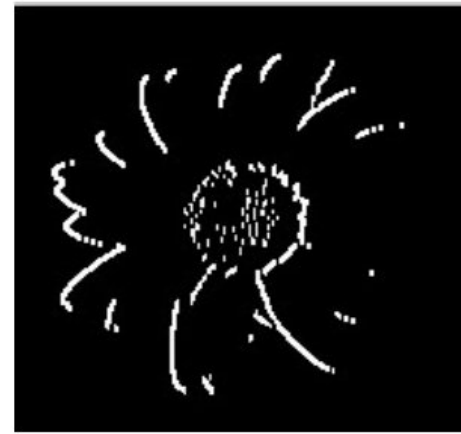
This represents a vertical edge between dark and bright regions.

Apply convolution:

$$\begin{aligned} &(-1 \times 10) + (0 \times 10) + (1 \times 200) \\ &+ (-2 \times 10) + (0 \times 10) + (2 \times 200) \\ &+ (-1 \times 10) + (0 \times 10) + (1 \times 200) \end{aligned}$$

Result:

$$\begin{aligned} &(-10 + 0 + 200) + (-20 + 0 + 400) + (-10 + 0 + 200) \\ &= 190 + 380 + 190 = 760 \end{aligned}$$



Applications

- Object boundary detection
- Road lane detection
- Face detection systems
- Medical image analysis

Edge Detection using Laplacian Filter

The **Laplacian filter** detects edges by computing the **second derivative of the image intensity**. It highlights regions where the intensity changes rapidly.

Unlike gradient methods, the Laplacian operator:

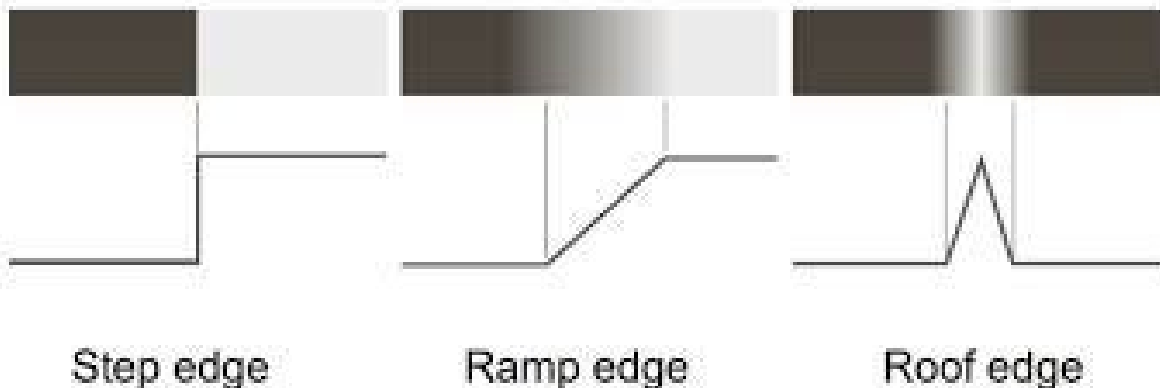
- Detects edges **in all directions**
- Uses a **single mask**
- Produces edges where intensity changes sharply.

Common Laplacian Mask

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

or

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$



Example

Consider the following 3×3 image region:

$$\begin{bmatrix} 10 & 10 & 10 \\ 10 & 100 & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

Apply Laplacian mask:

$$\begin{aligned} & (0 \times 10) + (-1 \times 10) + (0 \times 10) \\ & + (-1 \times 10) + (4 \times 100) + (-1 \times 10) \\ & + (0 \times 10) + (-1 \times 10) + (0 \times 10) \end{aligned}$$

Result:

$$\begin{aligned} & (-10) + (-10) + 400 + (-10) + (-10) \\ & = 360 \end{aligned}$$

Since the value is **large**, the center pixel is detected as part of an **edge region**.

Applications

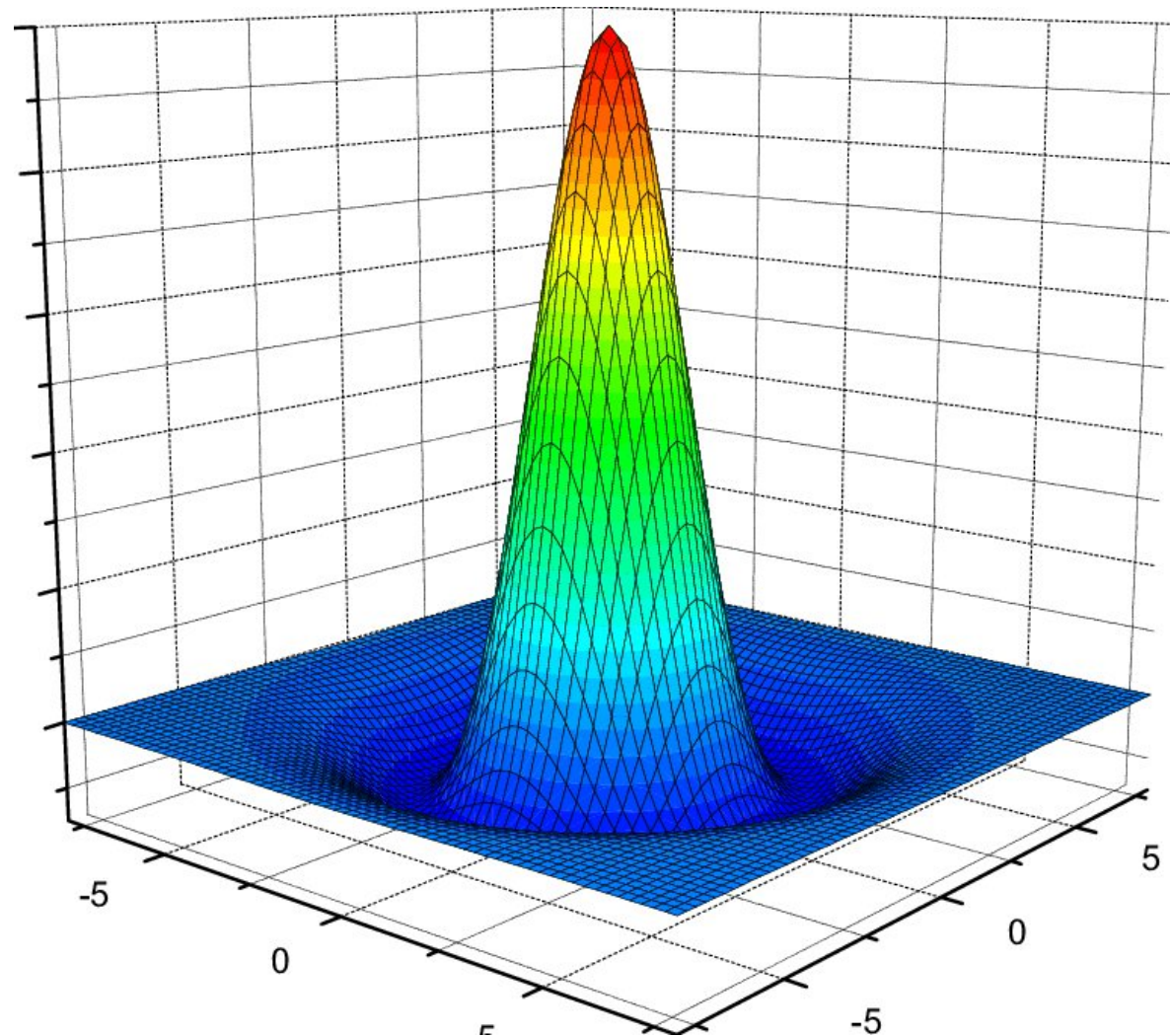
- Image sharpening
- Edge detection in medical images
- Satellite image analysis
- Feature extraction in computer vision

Mexican Hat Filter (Laplacian of Gaussian – LoG)

The Mexican Hat Filter is a filter used in image processing for edge detection and feature detection. It is also called the Laplacian of Gaussian (LoG) filter because it combines two operations:

- *Gaussian smoothing (to remove noise)*
- *Laplacian operator (to detect edges)*

The 3D shape of this filter looks like a Mexican sombrero hat, which is why it is called the Mexican Hat filter.



Key Idea

1. First smooth the image using a **Gaussian filter**.
2. Then apply the **Laplacian operator** to detect rapid intensity changes.
3. The filter highlights **edges and fine details** in the image.

Mathematically, it is expressed as:

$$LoG(x, y) = \nabla^2(G(x, y))$$

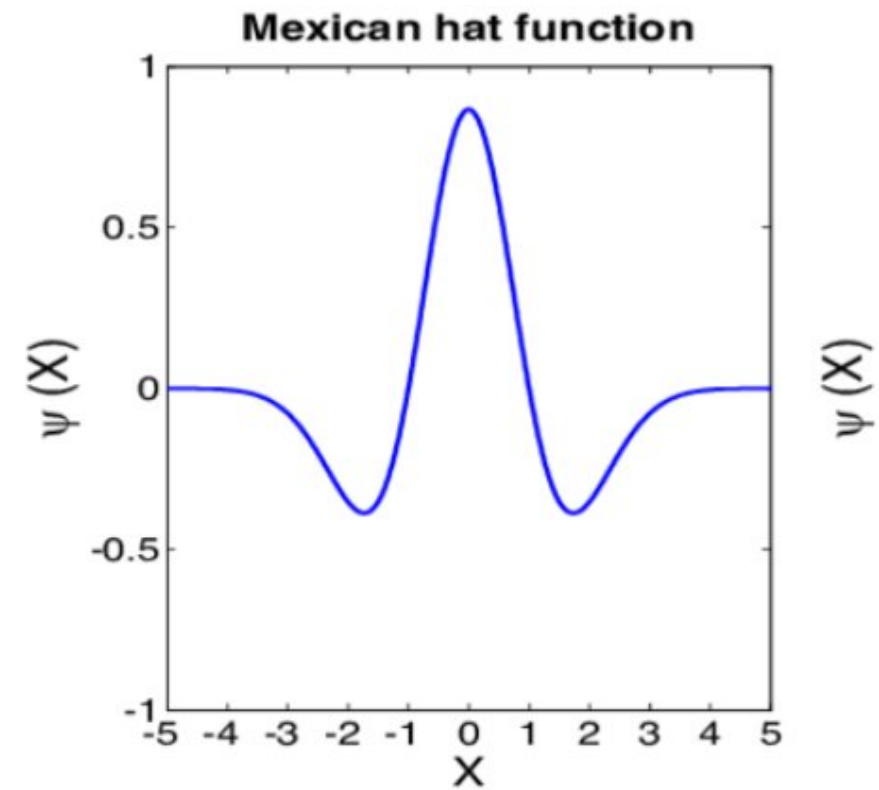
Where

- $G(x, y)$ = Gaussian function
- ∇^2 = Laplacian operator

The LoG function can be written as:

$$LoG(x, y) = \frac{x^2 + y^2 - 2\sigma^2}{\sigma^4} e^{-\frac{x^2 + y^2}{2\sigma^2}}$$

Where σ (**sigma**) controls the amount of smoothing.



Structure of Mexican Hat Filter (Matrix Form)

A typical 3×3 Mexican Hat filter mask is:

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Another commonly used 5×5 mask is:

$$\begin{bmatrix} 0 & 0 & -1 & 0 & 0 \\ 0 & -1 & -2 & -1 & 0 \\ -1 & -2 & 16 & -2 & -1 \\ 0 & -1 & -2 & -1 & 0 \\ 0 & 0 & -1 & 0 & 0 \end{bmatrix}$$

Characteristics

- Center value is **large positive**
- Neighboring values are **negative**
- Sum of mask values ≈ 0
- Highlights **edges and rapid intensity changes**

Applications of Mexican Hat Filter

1. Edge Detection

- Detects object boundaries in images.

2. Blob Detection

- Used to detect circular features (cells, stars, etc.).

3. Feature Extraction

- Identifying important patterns in images.

4. Computer Vision

- Object detection and image segmentation.

5. Medical Image Analysis

- Detect tumors or abnormal structures.

Example of Mexican Hat Filter (Matrix Calculation)

Step 1: Image Pixel Matrix

Suppose the image region is:

$$I = \begin{bmatrix} 10 & 10 & 10 \\ 10 & 50 & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

Since the output value is large, it indicates a strong intensity change (edge or feature) in the image.

Step 2: Mexican Hat Mask

$$H = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Step 3: Convolution Calculation

Output =

$$(10 \times -1) + (10 \times -1) + (10 \times -1)$$

$$+(10 \times -1) + (50 \times 8) + (10 \times -1)$$

$$+(10 \times -1) + (10 \times -1) + (10 \times -1)$$

$$= -10 - 10 - 10 - 10 + 400 - 10 - 10 - 10 - 10$$

$$= 320$$

$$Output = 320$$

Final Output

Final Output

$$Output = 320$$

Since the output value is **large**, it indicates a **strong intensity change (edge or feature)** in the image.

Example: Edge Detection using Mexican Hat Filter

After applying the Mexican Hat filter:

- Smooth regions → values near 0
- Edge regions → **high positive or negative values**
- Helps detect **object boundaries**

LoG with zero-crossing, sigma=1



LoG with zero-crossing, sigma=2



LoG with zero-crossing, sigma=3



LoG with zero-crossing, sigma=4

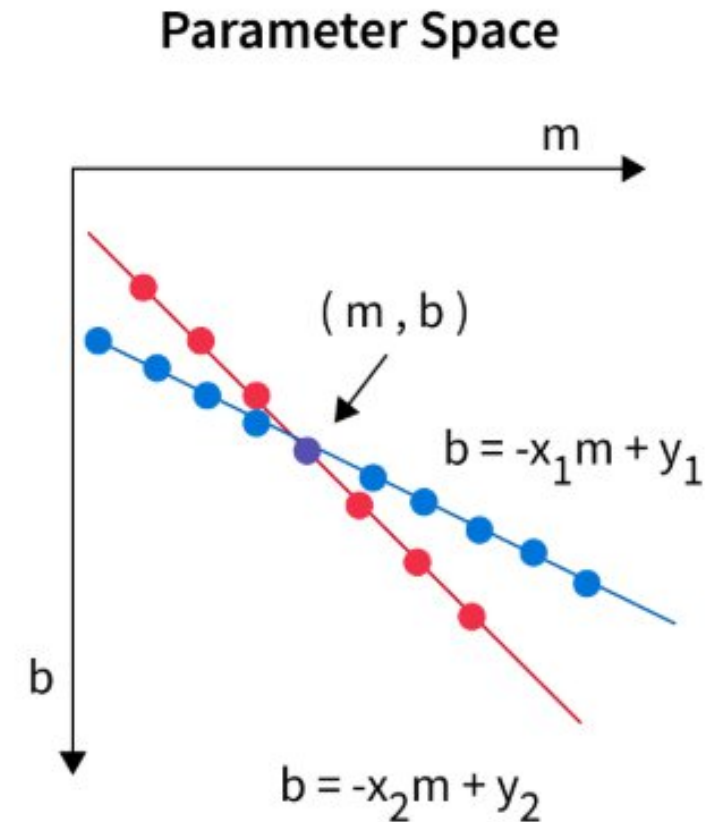
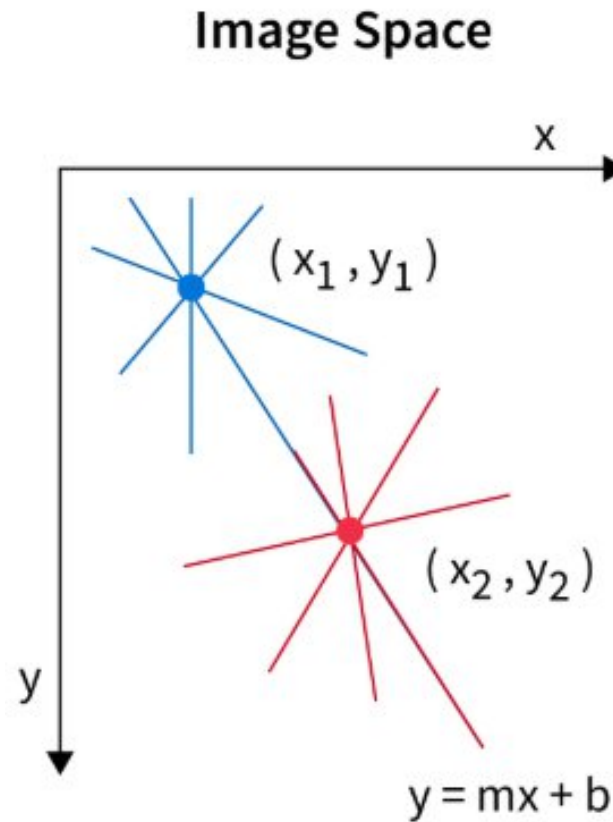


Hough Transform (Line Detection Technique)

The Hough Transform is a technique used in image processing to detect geometric shapes such as lines, circles, and curves in an image.

It is especially useful when shapes are incomplete, noisy, or partially hidden.

The main idea of the Hough Transform is to convert points from image space into parameter space and find intersections that represent a particular shape.

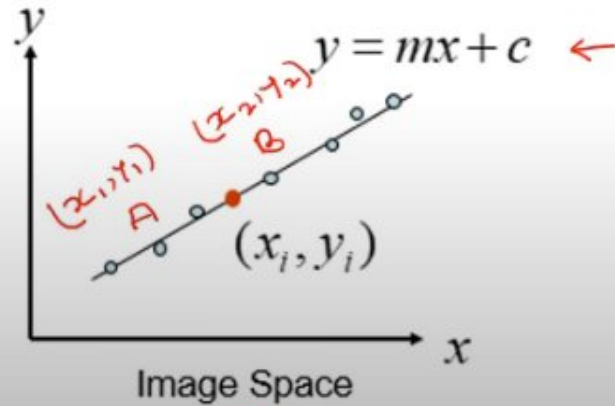


Equation of Line:

$$y = mx + c$$

Where,
 m = slope ✓
 c = intercept of the line ✓

Problem is that, Infinite lines can be drawn connecting these points



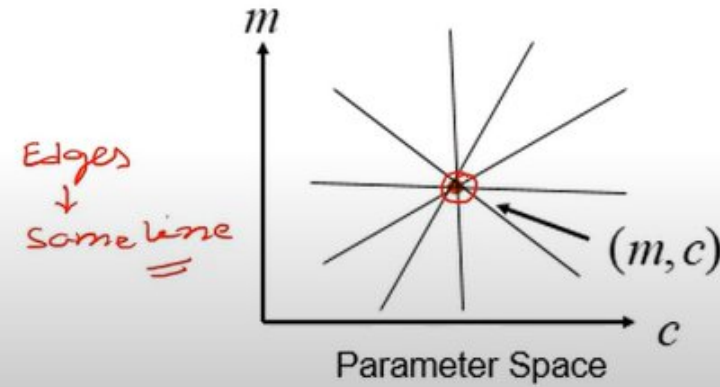
If A and B are two points connected by a line in spatial domain

They will be intersecting lines in Hough Space

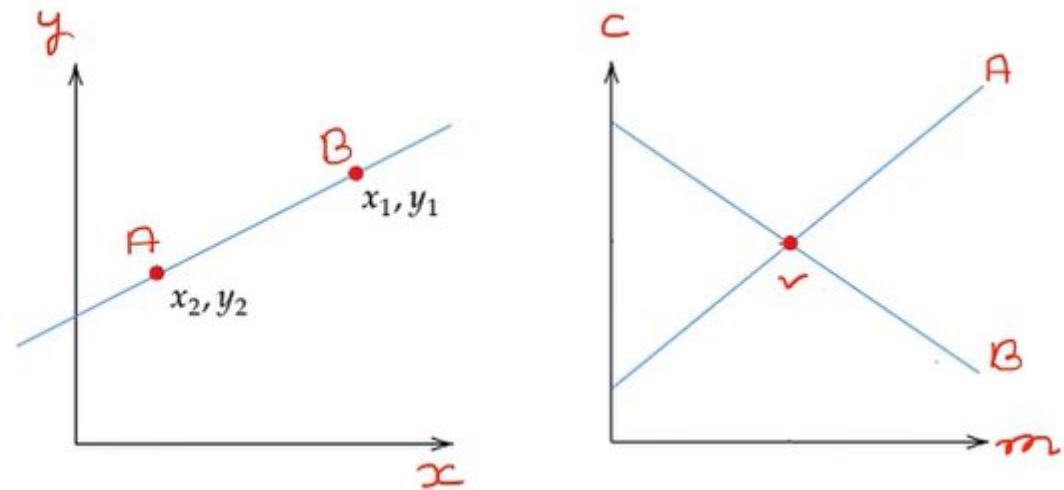
✓ Therefore an edge point in the x-y plane is transformed to the c-m

Point (x_i, y_i)

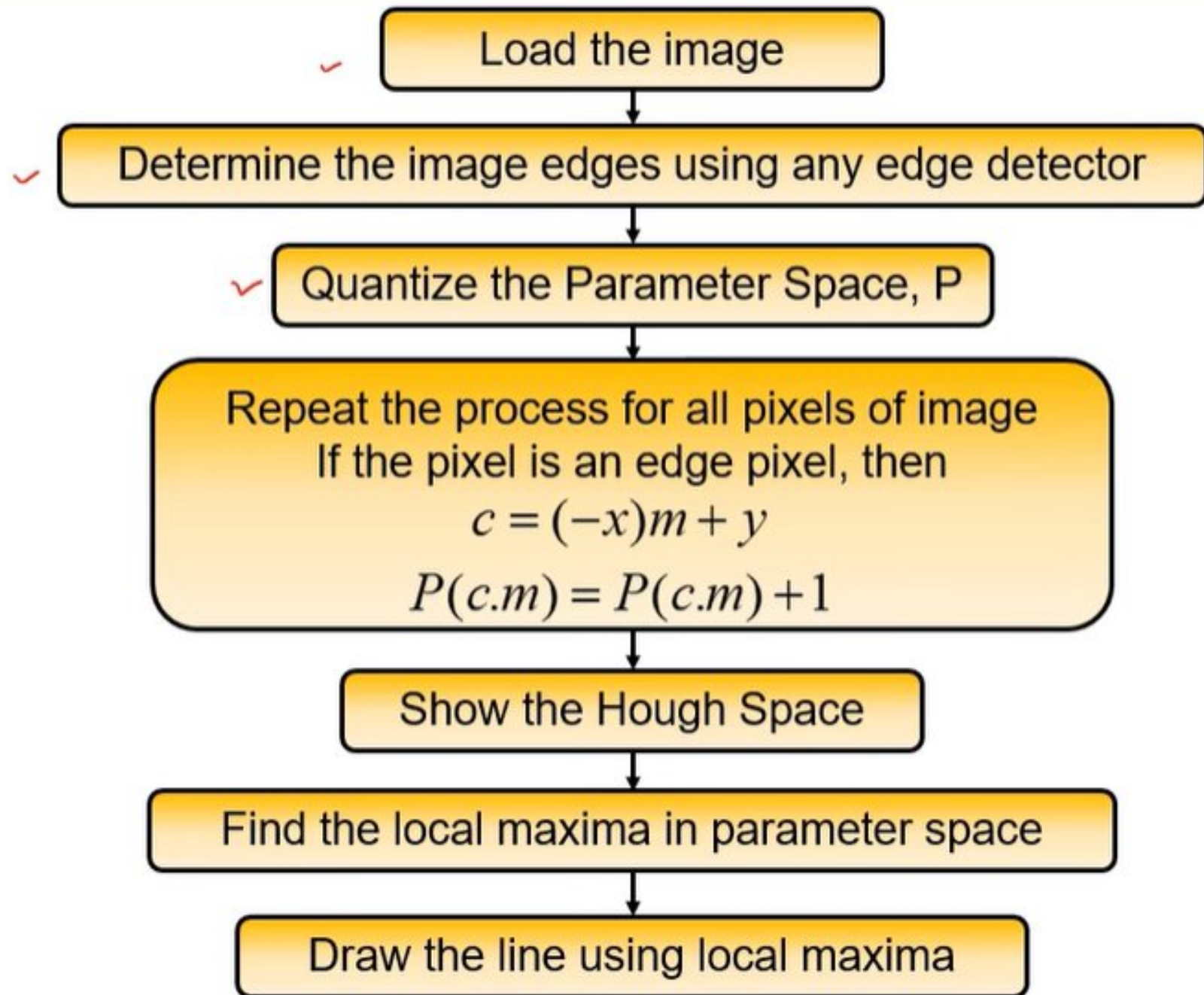
$$y_i = mx_i + c \quad \text{or} \quad c = -x_i m + y_i$$



Parameter space also called Hough Space



Algorithm



(i) For $(x, y) = (1, 2)$, $c = -m + 2$

if $c = 0$, $0 = -m + 2$

$m = 2$

if $m = 0$, $c = 2$

Thus, $(m, c) = (2, 2)$.

(ii) For $(x, y) = (2, 3)$, $c = -2m + 3$.

if $c = 0$, $0 = -2m + 3$

$2m = 3$

$m = 3/2 = 1.5$, $m = 1.5$

if $m = 0$, $c = 3$.

Thus, $(m, c) = (1.5, 3)$.

(iii) For $(x, y) = (3, 4)$, $c = -3m + 4$.

if $c = 0$, $0 = -3m + 4$

$3m = 4$

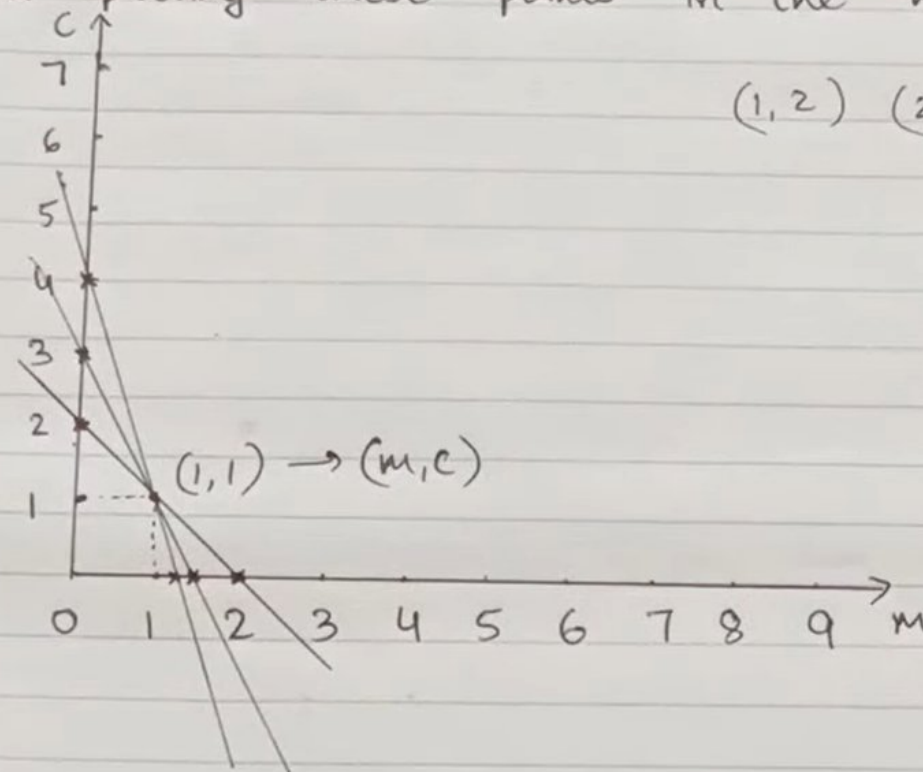
$m = 4/3 = 1.33$, $m = 1.33$

if $m = 0$, $c = 4$.

Thus $(m, c) = (1.33, 4)$.

$(m, c) = (2, 2)$, $(1.5, 3)$, $(1.33, 4)$.

On plotting these points in the m - c plane:



Original equation:

$$y = mx + c$$

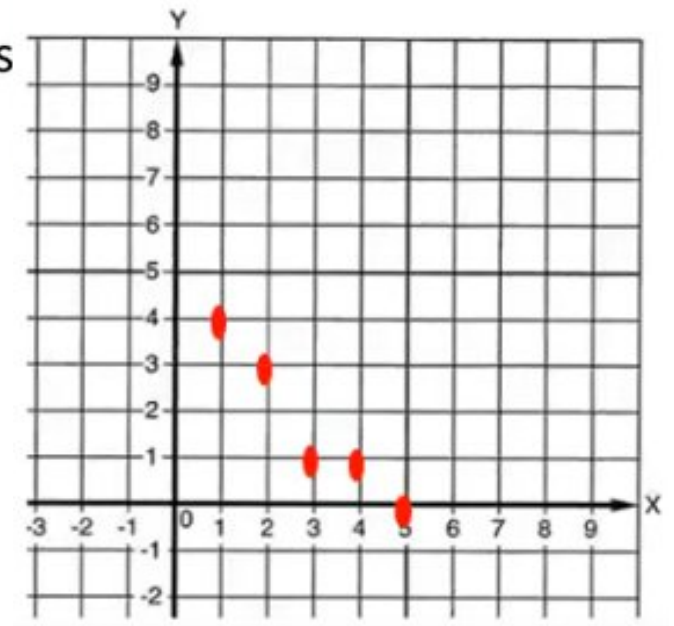
Substituting $(1, 1)$:

$y = x + 1 \rightarrow$ Final equation

Example1: Given 5 points use Hough transform to draw a line joining these points.

(1,4) (2,3) (3,1) (4,1) (5,0)

X	1	2	3	4	5
y	4	3	1	1	0

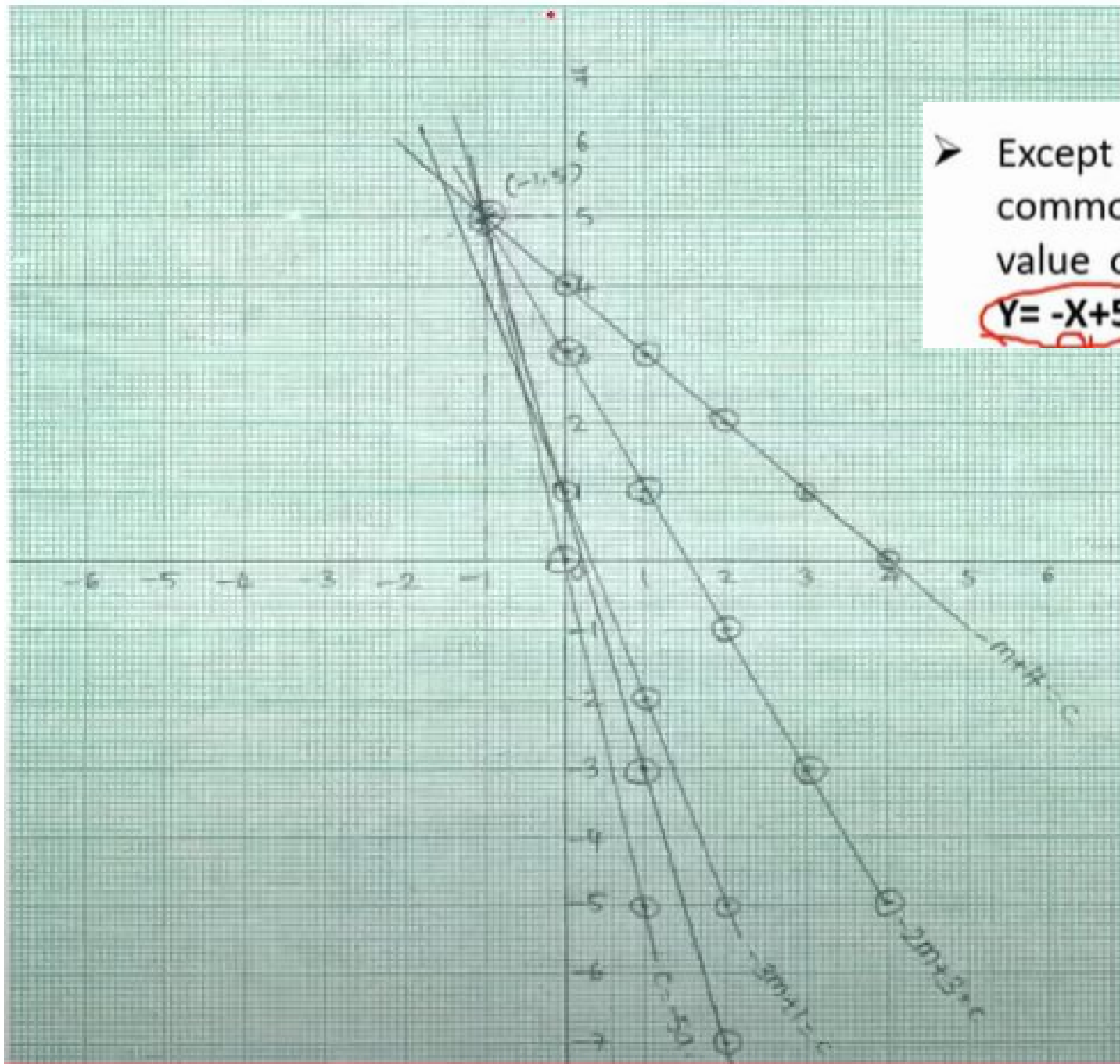


Equation of straight line $\rightarrow$

$$y = mx + c$$

$$\therefore c = y - mx$$

x	1	2	3	4	5
y	4	3	1	1	0
Eq.n of lines	$c = 4 - m$	$c = 3 - 2m$	$c = 1 - 3m$	$c = 1 - 4m$	$c = 0 - 5m$



- Except $(3, 1)$ all the remaining points meet at a common point $(-1, 5)$ where $m = -1$; $c = 5$. Hence this value of m and c are used to draw a line that is.

$$Y = -X + 5$$

Similarity-Based Techniques

Similarity-based segmentation groups pixels into regions based on **similar properties** like intensity, color, or texture. The main idea: pixels in the same region are similar, and pixels in different regions are dissimilar.

Thresholding is one of the simplest and most widely used similarity-based techniques.

Thresholding

Thresholding separates an image into **foreground** and **background** by comparing pixel intensity with a threshold value T .

The basic rule:

$$g(x, y) = \begin{cases} 1 & \text{if } f(x, y) \geq T \\ 0 & \text{if } f(x, y) < T \end{cases}$$

- $f(x, y)$ = pixel intensity at (x, y)
- $g(x, y)$ = output binary image

Thresholding can be classified into **Global, Local, and Adaptive** methods.

1. Global Thresholding

- **Definition:** Uses a single threshold value T for the entire image.
- **Best for:** Images with **uniform lighting**.
- **Example:**

Original image intensity:

Pixel	50	80	120	200
-------	----	----	-----	-----

Global threshold $T = 100$:

Pixel	50	80	120	200
-------	----	----	-----	-----

Output	0	0	1	1
--------	---	---	---	---

- Simple, fast, but fails if lighting varies across the image.

2. Local Thresholding (also called Dynamic Thresholding)

- **Definition:** Computes a threshold independently for each pixel based on the intensity of neighboring pixels.
- **Best for:** Images with **non-uniform illumination**.
- **Example:**

Consider a 3x3 neighborhood:

50	60	55
65	80	70
60	75	85

- Compute local average: $(50+60+55+65+80+70+60+75+85)/9 = 66.7$
- Pixel (80) > local threshold (66.7) → foreground (1)
- Works well when background intensity varies.

3. Adaptive Thresholding

- **Definition:** Threshold changes **per pixel** considering both **local mean and standard deviation**.
- **Formula (common):**

$$T(x, y) = \mu(x, y) + k \cdot \sigma(x, y)$$

- $\mu(x, y)$ = mean of neighborhood
- $\sigma(x, y)$ = standard deviation of neighborhood
- k = constant to fine-tune threshold
- **Advantage:** Handles **uneven illumination** better than simple local thresholding.
- **Example:**

For neighborhood: mean = 100, std = 15, $k = 0.5$

$$T = 100 + 0.5 * 15 = 107.5$$

- Pixel > 107.5 → foreground; else → background

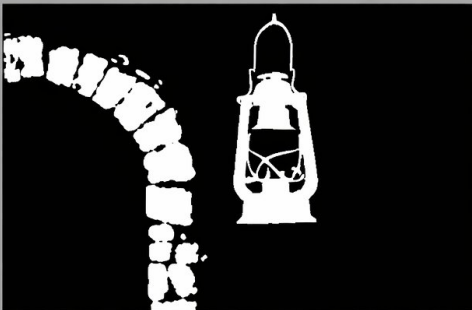
Summary Table

Method	Threshold Type	Pros	Cons
Global	Single T for entire image	Simple, fast	Fails with non-uniform illumination
Local	Threshold per pixel neighborhood	Handles varying lighting	Slower, sensitive to window size
Adaptive	Threshold depends on local mean & variance	Best for uneven illumination	More computation

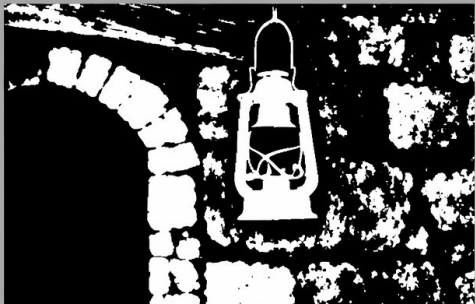
Original Image



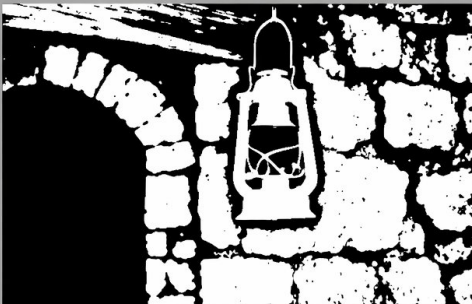
Global Thresholding



Local Thresholding



Adaptive Thresholding



- **Global Thresholding:** One cut-off; some regions lost under shadows.
- **Local Thresholding:** Each region segmented based on local intensity.
- **Adaptive Thresholding:** Best separation in all lighting conditions.

Region Growing Algorithm

Region Growing is a **region-based image segmentation** technique. It starts with **seed points** and grows regions by appending neighboring pixels that are **similar** based on a predefined criterion (e.g., intensity, color, or texture).

Steps:

1. **Select seed points:** Choose one or more starting pixels for each region.
2. **Check neighbors:** Examine neighboring pixels of the current region.
3. **Similarity test:** Add neighbors to the region if they satisfy a similarity criterion (e.g., intensity difference $< \text{threshold}$).
4. **Repeat:** Continue growing until no more pixels can be added.

Characteristics:

- **Input:** Image and seed points
- **Output:** Segmented regions
- **Advantages:** Preserves edges well, simple to implement
- **Disadvantages:** Sensitive to noise and seed selection

Example:

Consider a 5x5 grayscale image:

50	52	55	200	202
48	50	54	198	201
47	49	53	199	200
180	182	185	60	62
179	181	183	58	59

- Choose **seed at (0,0) = 50**.
- Threshold = ± 5 .
- Add neighbors: pixels between 45–55 get added to region 1.
- Another seed at (0,3) = 200 forms region 2 (neighbors between 195–205).

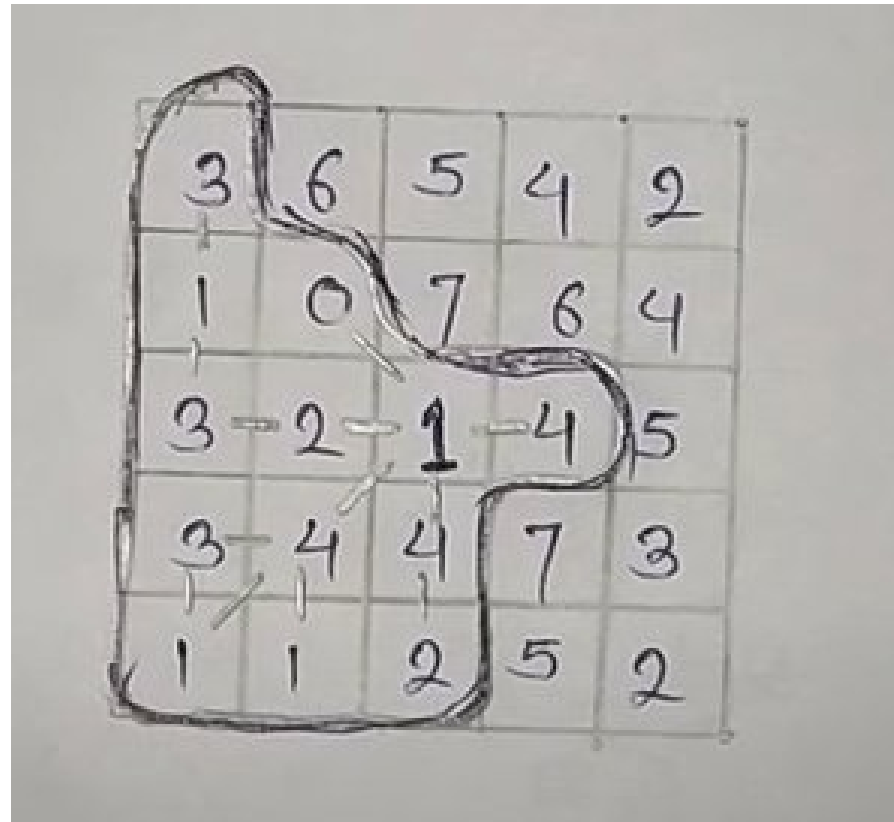
Final segmented regions matrix:

1	1	1	2	2
1	1	1	2	2
1	1	1	2	2
0	0	0	0	0
0	0	0	0	0

Result: Two main regions are segmented:

- *Region 1: Top-left cluster*
- *Region 2: Top-right cluster*
- *Bottom-left and bottom-right clusters may form additional regions depending on thresholds.*

Q. Apply the region Growing on the following image.
Assume the threshold value be ≤ 4 .



3	6	5	4	2
1	0	7	6	4
3	2	1	4	5
3	4	4	7	3
1	1	2	5	2

Problems with Region Growing Technique

1. Seed Dependency:

- Accuracy heavily relies on **manual seed selection**. Poor seed placement leads to incorrect segmentation.

2. Sensitivity to Noise:

- Noisy pixels can **leak** into regions or cause premature termination.

3. Threshold Sensitivity:

- Fixed threshold (T) may **overmerge** (too high) or **undersegment** (too low).

4. Computational Cost:

- Slow for large images due to **neighbor checks** for every pixel.

Practical Impact

- **Medical Imaging:** Missed tumor boundaries if seeds are misplaced.
- **Remote Sensing:** Merges distinct fields due to similar textures.

Solution: Often combined with **edge detection** or **morphological operations** to improve robustness.

Split and Merge Algorithm

Split and Merge Algorithm

Split and Merge is a **region-based segmentation technique** that combines **top-down (splitting)** and **bottom-up (merging)** strategies. It divides the image into regions based on **homogeneity criteria** (like intensity similarity).

Steps:

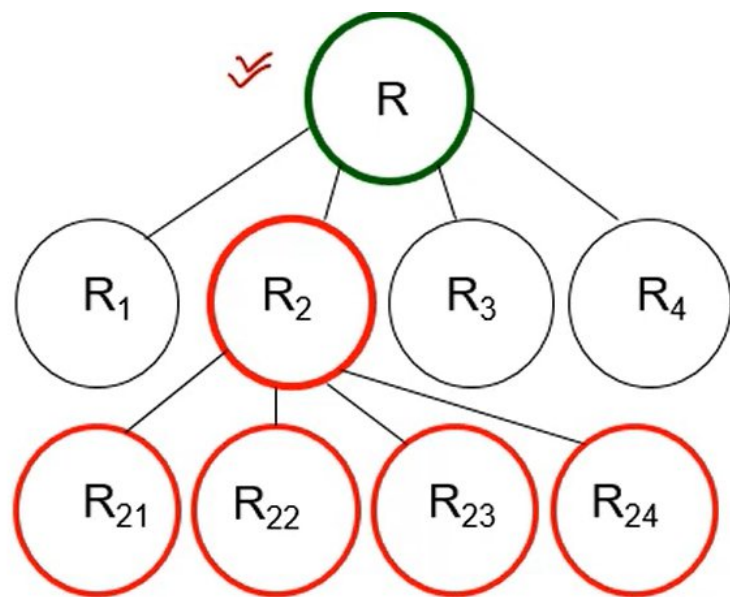
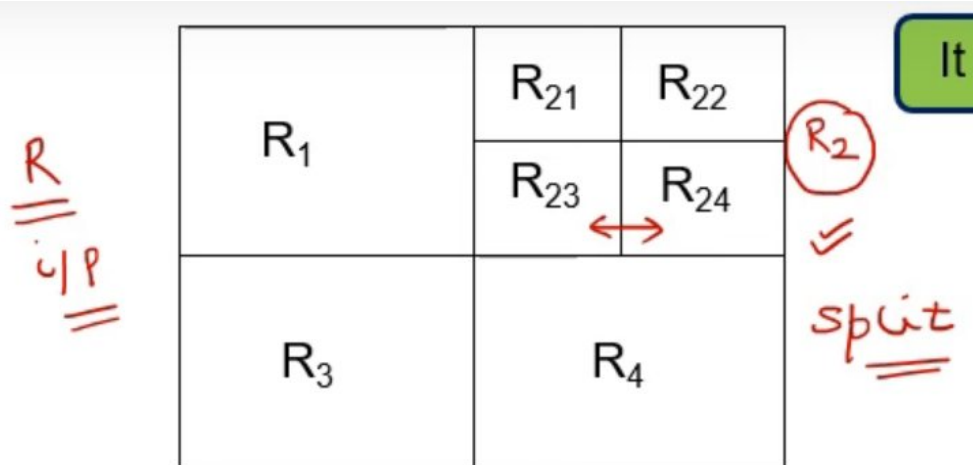
1. Splitting

- Start with the **whole image** as one region.
- If a region **does not satisfy the homogeneity criterion** (e.g., intensity variance too high), **split** it into 4 equal quadrants (quadtree decomposition).
- Repeat recursively until all regions are homogeneous.

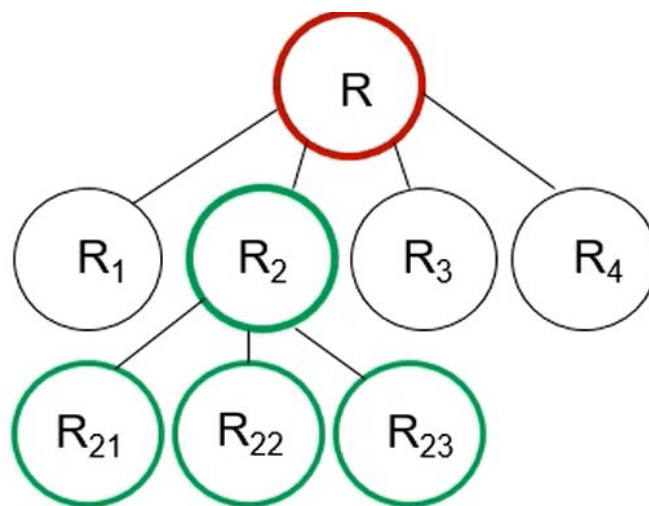
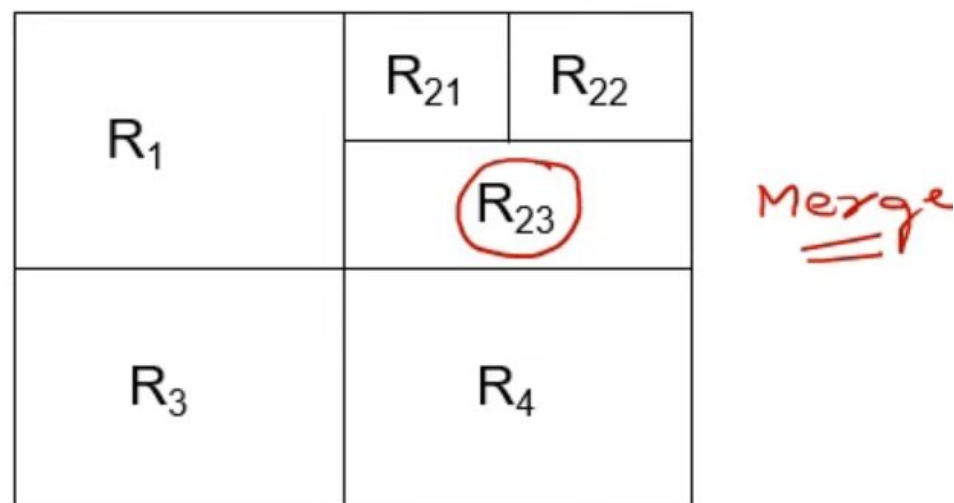
2. Merging

- After splitting, check **neighboring regions**.
- If neighboring regions are **similar**, **merge** them into a larger region.
- Continue until no further merging is possible.

It is also known as “**Divide and Conquer**” or “**Top Down**” approach



✓ Quadtree of Splitting



Quadtree of Splitting and Merging

Advantages:

- Handles complex images better than simple thresholding.
- Adaptive to varying intensity in regions.

Disadvantages:

- Can be computationally expensive.
- Needs a good homogeneity criterion.

Consider a **4x4 grayscale image**:

52	55	200	202
50	54	198	201
60	62	205	208
58	59	207	210

Step 1: Check Whole Image

- Threshold for homogeneity: intensity difference ≤ 10
- Max = 210, Min = 50 $\rightarrow$ difference = 160 $\rightarrow$ **not homogeneous** $\rightarrow$ Split

Step 2: Split into Quadrants

- Q1 (Top-left):

52	55
50	54

- Max-Min = 55-50 = 5 $\rightarrow$ homogeneous $\rightarrow$ keep

- Q2 (Top-right):

200	202
198	201

- Max-Min = 202-198 = 4 $\rightarrow$ homogeneous $\rightarrow$ keep

- Q3 (Bottom-left):

60	62
58	59

- Max-Min = 62-58 = 4 $\rightarrow$ homogeneous $\rightarrow$ keep

- Q4 (Bottom-right):

205	208
207	210

- Max-Min = 210-205 = 5 $\rightarrow$ homogeneous $\rightarrow$ keep

Step 3: Merge Neighboring Regions

- Merge Q1 and Q3? $\text{Max-Min} = 62-50 = 12 \rightarrow$ exceeds threshold $\rightarrow$ don't merge
- Merge Q2 and Q4? $\text{Max-Min} = 210-198 = 12 \rightarrow$ don't merge
- Merge Q1 & Q2? Adjacent? $\text{Max-Min} = 202-50 = 152 \rightarrow$ don't merge
- Merge Q3 & Q4? $\text{Max-Min} = 210-58 = 152 \rightarrow$ don't merge

Final Segmented Regions:

1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

- **Region 1:** Top-left ($\approx 50-55$)
- **Region 2:** Top-right ($\approx 198-202$)
- **Region 3:** Bottom-left ($\approx 58-62$)
- **Region 4:** Bottom-right ($\approx 205-210$)

✓ Summary

- **Split:** Divide large heterogeneous regions into smaller homogeneous ones.
- **Merge:** Combine neighboring homogeneous regions if they satisfy similarity.
- Handles both small and large regions effectively.

Example: For a given image, apply split and merge algorithm

Solution:

Threshold, $T \leq 3$

max. pixel = 7
min. pixel = 0 $\} 7 - 0 = 7$

condition does not satisfies $\rightarrow$ split =

Merge

7 4

(A) ✓ x

✓ x

(C) 3 0

6	5	6	6	7	7	6	6
6	7	6	7	5	5	4	7
6	6	4	4	3	2	5	6
5	4	5	4	2	3	4	6
0	3	2	3	3	2	4	7
0	0	0	0	2	2	5	6
1	1	0	1	0	3	4	4
1	0	1	0	2	3	5	4

7 2] 5 ≠ 3

(B) ✓

7 0] 7

(D) ✓

3 splitting

6	5	6	6	7	7	6	6
6	7	6	7	5	5	4	7
6	6	4	4	3	2	5	6
5	4	5	4	2	3	4	6
0	3	2	3	3	2	4	7
0	0	0	0	2	2	5	6
1	1	0	1	0	3	4	4
1	0	1	0	2	3	5	4

6	5	6	6	7	7	6	6
6	7	6	7	5	5	4	7
6	6	4	4	3	2	5	6
5	4	5	4	2	3	4	6
0	3	2	3	3	2	4	7
0	0	0	0	2	2	5	6
1	1	0	1	0	3	4	4
1	0	1	0	2	3	5	4

max.(1) - min.(2)
max.(2) - min.(1)
 $T \leq 3$

Region Split and Merge Example

- Apply Region Split on following image. Assume that threshold value be ≤ 4 .

5	6	6	6	7	7	6	6
6	7	6	7	5	5	4	7
6	6	4	4	3	2	5	6
5	4	5	4	2	3	4	6
0	3	2	3	3	2	4	7
0	0	0	0	2	2	5	6
1	1	0	1	0	3	4	4
1	0	1	0	2	3	5	4

- Step 1: Identify Max and Min pixel value from the whole image

- Max = 7
- Min = 0
- Max-Min= 7
- $7 \leq 4$ (Condition false)
 - Split R image into R1a,R2b,cR3,R4d

- Step 2: Compute for R1a region. Min and Max

- Max=7
- Min=4
- Max-Min=7-4=3
- $3 \leq 3$ (Condition true)
 - No need to split R1a

- Step 3: compute for R3c region. Min and Max

- Max=3
- Min=0
- Max-Min=3-0=3
- $3 \leq 3$ (Condition true)
 - No need to split R3c

- Step 4: Compute for R2b region. Min and Max

- Max=7
- Min=2
- Max-Min=7-2=5
- $5 \leq 3$ (Condition false)
 - Split R2b into R2b1,R2b2,R2b3,R2b4.

- Step 6: Compute for R4d region. Min and Max

- Max=7
- Min=0
- Max-Min=7-0=7
- $7 \leq 3$ (Condition false)
 - Split R4d into R4d1,R4d2,R4d3,R4d4.

- Step 7: Compute for R4d1,R4d2,R4d3,R4d4 region. Min and Max

- For all R4d1,R4d2,R4d3,R4d4
Condition have satisfied so no need to splitting.

5	6	6	6	7	7	6	6
6	7	6	7	5	5	4	7
6	6	4	4	3	2	5	6
5	4	5	4	2	3	4	6
0	3	2	3	3	2	4	7
0	0	0	0	2	2	5	6
1	1	0	1	0	3	4	4
1	0	1	0	2	3	5	4

R1a				R2b			
5	6	6	6	7	7	6	6
6	7	6	7	5	5	4	7
6	6	4	4	3	2	5	6
5	4	5	4	2	3	4	6
0	3	2	3	3	2	4	7
0	0	0	0	2	2	5	6
1	1	0	1	0	3	4	4
1	0	1	0	2	3	5	4
R3c				R4d			

```
[
  [10, 10, 50, 50],
  [10, 10, 50, 50],
  [80, 80, 20, 20],
  [80, 80, 20, 20]
]
```

- A region is homogeneous if **max intensity – min intensity ≤ 20** .

Step 1: Initial Split (Quad-Tree Partitioning)

1. Check Entire Image (4×4):

- Min = 10, Max = 80 → **80 – 10 = 70 > 20** → **Not homogeneous**.
- **Split into 4 equal 2×2 blocks.**

2. Check Each 2×2 Block:

- **Top-Left Block:** [10, 10, 10, 10] → Max–Min=0 ≤ 20 → **Homogeneous (Keep)**.
- **Top-Right Block:** [50, 50, 50, 50] → Max–Min=0 ≤ 20 → **Homogeneous (Keep)**.
- **Bottom-Left Block:** [80, 80, 80, 80] → Max–Min=0 ≤ 20 → **Homogeneous (Keep)**.
- **Bottom-Right Block:** [20, 20, 20, 20] → Max–Min=0 ≤ 20 → **Homogeneous (Keep)**.

No further splits needed!

Step 2: Merge Adjacent Homogeneous Regions

1. Try Merging Top-Left (10) + Top-Right (50):

- Combined Min=10, Max=50 → 50–10=40 > 20 → **Cannot merge**.

2. Try Merging Top-Left (10) + Bottom-Left (80):

- Combined Min=10, Max=80 → 80–10=70 > 20 → **Cannot merge**.

3. Try Merging Top-Right (50) + Bottom-Right (20):

- Combined Min=20, Max=50 → 50–20=30 > 20 → **Cannot merge**.

```
[
  [1, 1, 2, 2], # 1=Region(10), 2=Region(50)
  [1, 1, 2, 2],
  [3, 3, 4, 4], # 3=Region(80), 4=Region(20)
  [3, 3, 4, 4]
]
```

Key Problems Solved by Split & Merge

- **Over-segmentation:** Avoided by merging similar regions.
- **Under-segmentation:** Avoided by splitting non-homogeneous blocks.

Real-World Use Case

- **Satellite Imagery:** Segmenting fields/forests by splitting irregular areas and merging similar textures.
- **Medical Imaging:** Isolating tumors by splitting heterogeneous regions.

Advantage: No seed dependency (unlike Region Growing).

Disadvantage: Computationally expensive for large images.

Explain how can you apply segmentation in line defection

Line Detection Using Segmentation

Line detection is a type of **feature detection**, often used in image processing for:

- Detecting **edges and boundaries** of objects
- Recognizing **roads, lanes, or patterns**
- Preparing images for **OCR or CAD applications**

Segmentation helps in line detection by **isolating regions of interest**, making it easier to apply line detection algorithms like **Hough Transform** or **edge-based methods**.

Step 1: Preprocessing

- Convert the image to **grayscale** (if not already).
- Apply **noise reduction** using filters like **Gaussian**.

Step 2: Segmentation

Segmentation can be used to **highlight linear regions**:

1. **Thresholding**: Separate dark lines from bright background.
 - Example: A scanned document has black lines on white paper.
 - Apply **global or adaptive thresholding** to create a **binary image**:
 - Lines $\rightarrow$ 1 (white)
 - Background $\rightarrow$ 0 (black)
2. **Region Growing**: Can be used if lines are broken or irregular.
 - Start from seed points on the line.
 - Grow region by adding connected pixels that match line intensity.
3. **Split and Merge**: Useful for large images with multiple lines.
 - Split image into quadrants.
 - Merge regions that form linear shapes.

Step 3: Edge Detection

- Use **gradient-based methods** (Sobel, Prewitt) to find edges in segmented regions.
- Output: **binary edge map** highlighting line edges.

Step 4: Line Detection

- Apply **Hough Transform** on the binary image:
 - Convert edge pixels (x, y) to Hough space (ρ, θ)
 - Detect lines as peaks in Hough space
- Output: Lines are overlaid on original image.

Step	Role of Segmentation	Output
Thresholding	Separates line pixels from background	Binary image
Region Growing	Connects broken line pixels	Connected line segments
Split & Merge	Handles large, multiple line regions	Homogeneous linear regions
Edge Detection	Detects line boundaries	Edge map
Hough Transform	Detects straight lines from edges	Line coordinates

Past Questions on Image Segmentation

- Explain how Hough transform is useful in line detection?
- Explain the region growing technique for image segmentation.
- Explain the adaptive thresholding and region split and merge techniques for image segmentation.(5+5)
- Explain in detail the region growing techniques for image segmentation. List the problems associated with region growing technique.(4+2)
- Explain in detail the region split and merge techniques for image segmentation. List the problems associated with region split and merge technique.(4+2)
- What is a gradient filter? Explain the Sobel gradient filter in detail along with its algorithm for implementation.(1 + 5)
- Explain the segmentation by thresholding. Explain how can you apply segmentation in line detection.
- How edges are detected using the gradient operators? Explain with suitable example.